Competitive Programming Ad-Hoc Sheet

Contents

1 Number Theory **2**

1.1 Basic Counting, Legendre & Harmonic Partitioning . . . 2

1.2 Coprime Properties & GCD Identities . . . 3

1.3 Prime Factorization . . . 3

1.4 Modular Arithmetic . . . 4

2 Combinatorics **5**

3 Mathematical Tricks **6**

4 Array & Invariant Transformations **7**

5 Bit Manipulation **9**

5.1 Range Bit Counting . . . 10

6 Bitwise Operations over Subsequences **11**

6.1 XOR Operations . . . 11

6.2 OR Operations . . . 12

6.3 AND Operations . . . 13

7 Bitwise Operations over Subarrays **14**

7.1 XOR Operations . . . 14

7.2 OR Operations . . . 15

7.3 AND Operations . . . 16

8 Range Queries & Bitwise Operations **17**

8.1 Lazy Propagation Updates . . . 17

8.2 Querying Subsequences over a Range $[L, R]$ . . . 17

8.3 Querying Subarrays over a Range $[L, R]$ . . . 17

9 String Algorithms **19**

10 Greedy & Poset Principles **21**

11 Chess Problems **21**

12 Geometry **22**

1 Number Theory

1.1 Basic Counting, Legendre & Harmonic Partitioning

Factorials, Squares & Legendre's Formula

Trailing Zeros in $n!$:

$$\left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{n}{5^2} \right\rfloor + \left\lfloor \frac{n}{5^3} \right\rfloor + \dots \text{ until } 0$$

Legendre's Formula (Prime Power in $n!$):
The exact exponent $E_p(n!)$ of prime p dividing $n!$ is:

$$E_p(n!) = \sum_{k=1}^{\infty} \left\lfloor \frac{n}{p^k} \right\rfloor = \frac{n - S_p(n)}{p-1}$$

where $S_p(n)$ is the sum of digits of n when written in base p.
Number of perfect squares $\leq n$:

$$\lfloor \sqrt{n} \rfloor$$

Maximum Number of Divisors $d(n)$

Worst-case bounds for iterating over all divisors:

- $n \leq 10^9 \implies d(n) \leq 1344$ (at $n = 735{,}134{,}400$)
- $n \leq 10^{12} \implies d(n) \leq 6720$ (at $n = 963{,}761{,}198{,}400$)
- $n \leq 10^{18} \implies d(n) \leq 103680$ (at $n = 897{,}612{,}484{,}786{,}617{,}600$)

Square Root Harmonic Partitioning $O(\sqrt{N})$

```
for (ll l = 1, r; l <= n; l = r + 1)
{
    ll val = n / l;
    r = n / val;
}
```

$\lfloor n/i \rfloor$ takes at most $2\sqrt{n}$ distinct values over $1 \leq i \leq n$. For each block $[l, r]$, the value `val` $= \lfloor n/l \rfloor$ is constant, and $r = \lfloor n/\text{val} \rfloor$ is the last index sharing that value.
Use case: Compute $\sum_{i=1}^{n} f(i) \cdot \lfloor n/i \rfloor$ with $O(\sqrt{n})$ block evaluations: given prefix sums F of f, block $[l, r]$ contributes `val` $\cdot \big(F(r) - F(l-1)\big)$.

Fibonacci Sequence

Range of Fibonacci numbers in 64 bits: with $F_1 = F_2 = 1$, F_{92} is the last Fibonacci number that fits in a signed 64-bit integer.
Check if N is a Fibonacci number:
A positive integer N is a Fibonacci number if and only if either $5N^2 + 4$ or $5N^2 - 4$ is a perfect square (beware that $5N^2$ overflows signed 64-bit for $N \gtrsim 1.36 \cdot 10^9$ (unsigned: $1.9 \cdot 10^9$); use `__int128`).

1.2 Coprime Properties & GCD Identities

Euler's Totient & Coprimes

Sum of numbers $< n$ that are coprime to n (for $n > 1$)**:**

$$\sum_{\substack{k=1 \\ \gcd(k,n)=1}}^{n-1} k = \frac{n \cdot \varphi(n)}{2}$$

Count numbers $\leq m$ that are coprime to n:

$$\Phi(n, m) = \sum_{d \mid \text{rad}(n)} \mu(d) \left\lfloor \frac{m}{d} \right\rfloor$$

where $\text{rad}(n) = p_1 \cdot p_2 \cdots p_k$ (product of distinct prime factors) and $\mu(d)$ is the Möbius function.
Count ordered coprime pairs (a, b) such that $a \cdot b = m$:

$$2^{\omega(m)}$$

where $\omega(m)$ is the number of distinct prime factors of m.

Algebraic GCD Identities

- **Exponential GCD** ($a > 1$): $\gcd(a^n - 1, a^m - 1) = a^{\gcd(n,m)} - 1$
- **Mersenne / Power Divisibility:** $(2^x - 1) \mid (2^n - 1) \iff x \mid n$ (more generally, for any base $b \geq 2$, $(b^x - 1) \mid (b^n - 1) \iff x \mid n$, as a direct corollary of Exponential GCD).
- **Fibonacci GCD:** $\gcd(F_n, F_m) = F_{\gcd(n,m)}$
- **Euclidean Difference:** For $a > b > 0$, $\gcd(a, b) = \gcd(a - b, b) = \gcd(a \bmod b, b)$.

Frobenius Coin Problem (Chicken McNugget Theorem)

For two coprime positive integers a and b ($\gcd(a, b) = 1$):

> **Largest unrepresentable integer:** $a \cdot b - a - b$

- **Total unrepresentable positive integers:** $\frac{(a-1)(b-1)}{2}$
- Every integer $k > a \cdot b - a - b$ can be expressed as $k = ax + by$ for non-negative integers $x, y \geq 0$.

1.3 Prime Factorization

Count Distinct Prime Factors $\omega(m)$ $O(N)$

```
for (int i = 2; i <= n; i++)
{
    int q = i / lpf[i];
    omega[i] = omega[q] + (lpf[i] != lpf[q]);
}
```

Explanation:

- `lpf[i]`: Smallest prime factor of i (from Sieve).
- `q`: Removing one factor of the smallest prime.
- Logic: If the smallest prime of i is different from the smallest prime of q, we found a new distinct factor.

Totient Divisibility

Count number of x such that $\varphi(x) \mid x$ and $x \le n$:
Count $x = 1$, and numbers of the form $x = 2^a \cdot 3^b$ with $a \ge 1$, $b \ge 0$ and $x \le n$. (Reason: $x/\varphi(x) = \prod_{p|x} \frac{p}{p-1}$ is an integer only if the primes of x form $\emptyset$, $\{2\}$ or $\{2, 3\}$.)
(Pure powers of 3 are excluded: e.g. $\varphi(3) = 2 \nmid 3$.)

Count $x = 2^a \cdot 3^b \le n, a \ge 1, b \ge 0$ $O(\log^2 n)$

```
ll cnt = 1; // x = 1
for (ll p2 = 2; p2 <= n; p2 <<= 1)
{
    for (ll p3 = p2; p3 <= n; )
    {
        cnt++;
        if (p3 > n / 3)
            break;
        p3 *= 3;
    }
}
```

1.4 Modular Arithmetic

Wilson's Theorem

For any prime p:

$$(p-1)! \equiv -1 \pmod{p} \implies (p-1)! \equiv p-1 \pmod{p}$$

2 Combinatorics

Binomial Identities

- $\sum_{i=1}^{n} i\binom{n}{i} = n \cdot 2^{n-1}$
- $\sum_{\substack{0 \le i \le n \\ i \text{ even}}} \binom{n}{i} = 2^{n-1} \quad (\text{for } n \ge 1)$
- $\sum_{\substack{0 \le i \le n \\ i \text{ odd}}} \binom{n}{i} = 2^{n-1} \quad (\text{for } n \ge 1)$
- **Lucas Theorem (Parity):** $\sum_{i=0}^{n} \sum_{j=0}^{i} \left(\binom{i}{j} \bmod 2 \right) = \sum_{i=0}^{n} 2^{\text{popcount}(i)}$
- $\sum_{i=0}^{2^n-1} 2^{\text{popcount}(i)} = \sum_{k=0}^{n} \binom{n}{k} 2^k = 3^n$

$\sum_{i=0}^{n} \sum_{j=0}^{i} \left(\binom{i}{j} \bmod 2 \right)$ $O(\log n)$

```
Mint f(ll n)
{
    if (n == 0)
        return 1_m;
    int k = __lg(n);
    return pow3[k] + 2 * f(n - (1LL << k));
}
```

Stars and Bars

- **Non-negative solutions** to $x_1 + \cdots + x_k = n$: $\binom{n+k-1}{k-1}$
- **Positive integer solutions** ($x_i \ge 1$): $\binom{n-1}{k-1}$

Catalan Numbers (C_n)

$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$

- **Recurrence:** $C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}$
- **Parity:** C_n is odd $\iff n = 2^k - 1$

Catalan Number $O(1)$

```
Mint catalan(int n)
{
    return (nCr(2 * n, n) - nCr(2 * n, n + 1));
}
```

Derangements (Permutations with $p_i \neq i$) $O(N)$

```
vector<Mint> D(max(2, n + 1));
D[0] = 1, D[1] = 0;
for (int i = 2; i <= n; i++)
    D[i] = Mint(i - 1) * (D[i - 1] + D[i - 2]);
```

3 Mathematical Tricks

Median Trick

To check if a subarray has **median** $\geq m$ (for even length, the median is the lower middle element; use sum ≥ 0 for the upper one):

1. Assign $+1$ if $a[i] \geq m$, -1 otherwise.
2. If the sum of the subarray > 0, the median is at least m.

Minimum Check Trick

To verify whether the minimum value in a range is at least k:

1. Replace each element with 1 if $a[i] \geq k$, and 0 otherwise.
2. The range minimum is $\geq k$ iff the sum over that range equals the length of the range.

For a grid, check whether the sum over the submatrix equals its area.

Maximize Mismatched Items

For quantities $a[1 \dots n]$: The maximum number of disjoint pairs of items of different kinds is

$$\min\left(\left\lfloor \frac{\text{total}}{2} \right\rfloor, \text{total} - \max(a)\right), \qquad \text{total} = \sum_i a[i].$$

4 Array & Invariant Transformations

Kadane's Circular Maximum Subarray Sum

The maximum contiguous sum in a circular array A of size n is:

$$\max\left(\text{maxSubarraySum}(A),\quad \sum_{i=1}^{n} A[i] - \text{minSubarraySum}(A)\right)$$

Note: If all elements are negative, return maxSubarraySum(A) to avoid picking an empty subarray.

Circular Maximum Subarray Sum $O(N)$

```
ll total = 0, curMax = 0, maxSub = -2e18, curMin = 0, minSub = 2e18;
for (const ll &x : vc)
{
    total += x;
    curMax = max(x, curMax + x);
    maxSub = max(maxSub, curMax);
    curMin = min(x, curMin + x);
    minSub = min(minSub, curMin);
}
ll res = (maxSub < 0) ? maxSub : max(maxSub, total - minSub);
```

Difference Array for Range Updates

When an operation allows adding to an interval $[l, r]$:

- Transform A to difference array $D[i] = A[i] - A[i-1]$.
- Adding x to $[l, r]$ updates **only two positions**: $D[l]$ and $D[r+1]$.
- Reduces range manipulations to pairing and matching indices on the difference array.

Range Add & Range GCD via Difference Array

GCD does not distribute over addition, but by the Euclidean property:

$$\gcd(A[l], A[l+1], \ldots, A[r]) = \gcd\left(A[l],\ \gcd_{i=l+1}^{r} |D[i]|\right)$$

Range Add & Range GCD $O(\log N \log(\max A))$

```
// Update: Add x to range [l, r]  (arrays are 1-indexed, D has size n + 2)
bit.update(l, x);
bit.update(r + 1, -x);
segTree.update(l, D[l] += x);
if (r + 1 <= n)
    segTree.update(r + 1, D[r + 1] -= x);

// Query: Range GCD on [l, r]
auto slv = [&](int l, int r)
{
    ll f = A[l] + bit.query(l);
    if (l == r)
        return abs(f);
    return gcd(f, segTree.query(l + 1, r));
};
```

Permutation Inversion Parity & Reversals

- An **adjacent swap** strictly changes the number of inversions by ± 1 (toggles parity).
- A **3-element cyclic shift** $(a, b, c) \to (b, c, a)$ preserves inversion parity.
- **Reversing a Subsegment of length k:** Reverses the order of all $\binom{k}{2}$ pairs. If x of those pairs were inverted, there are now $\binom{k}{2} - x$ inversions. The change in inversions is $\binom{k}{2} - 2x \equiv \binom{k}{2} \pmod 2$:

$$\text{newParity} = \left(\text{oldParity} + \frac{k(k-1)}{2}\right) \bmod 2$$

- **Solvability Invariant** ($n \geq 3$): Using *only* 3-element cyclic shifts, a target permutation is reachable if and only if its inversion parity equals the initial parity. With adjacent swaps every permutation is reachable, and the number of swaps used has the same parity as (initial parity $\oplus$ target parity).

Range Reversal Inversion Parity Queries $O(N \log N + Q)$

```
ll inv{};
for (int i{}; i < n; i++)
{
    inv += bit.rangeQuery(vc[i] + 1, n);
    bit.update(vc[i], 1);
}
ll par = (inv & 1);

// Query: Reverse range [l, r]
ll k = r - l + 1;
par = (par + k * (k - 1) / 2) & 1;
```

15-Puzzle / Grid Inversion Invariant

For an $N \times N$ sliding tile grid with a blank cell (inversions are counted over the tiles only, ignoring the blank):

- If N is **odd**: Grid is solvable $\iff$ inversion count of flattened tiles is **even**.
- If N is **even**: Grid is solvable $\iff$ (inversions+row of the blank counted from the bottom, with the bottom row $= 1$) is **odd**.

5 Bit Manipulation

Submask Enumeration of All Masks $O(3^N)$

```
for (int mask = 0; mask < (1 << n); mask++)
{
    for (int sub = mask; sub > 0; sub = (sub - 1) & mask)
    { // process non-empty submask ‘sub‘ of ‘mask‘ (handle sub = 0 separately) }
}
```

Bitset & Number Rotations $O(1)$

```
// (a shift by BITS is undefined behaviour; C++20 provides std::rotl / std::rotr)
auto rotl = [&](unsigned long long x, int k) { return (x << k) | (x >> (BITS - k)); };
auto rotr = [&](unsigned long long x, int k) { return (x >> k) | (x << (BITS - k)); };
```

> For `std::bitset`, left and right shifts (`<<`, `>>`) fill with zeros, so combine them identically (for $0 < k <$ `BITS`): `(b << k) | (b >> (BITS - k))`.

XOR Identities & Bitwise Monotonicity

- $a \oplus b = a + b - 2 \cdot (a \,\&\, b)$
- $a \oplus b = 2 \cdot (a \mid b) - (a + b)$
- **SUM = OR Trick:** $a_l + \cdots + a_r = a_l \mid \cdots \mid a_r$ holds $\iff$ no two elements share a set bit (pairwise AND $= 0$).
- Hence a segment with SUM = OR contains at most 32 (or 64) non-zero elements, allowing brute force over the non-zero elements.
- **OR/AND/GCD Monotonicity:** For any fixed left endpoint l, extending r in $\mathrm{OR}(l, r)$, $\mathrm{AND}(l, r)$, or $\mathrm{GCD}(l, r)$ can change values at most $\approx \log_2(\max A)$ times.

Counting Values with XOR & Addition for a Fixed x

Let $W = \lfloor \log_2 x \rfloor + 1$ be the bit-width of x (in C++: `__lg(x) + 1`).

- **Count of a such that $a \oplus x > x$ $(0 < a < x)$:**

$$2^W - 1 - x$$

  Reason: Any $y \in (x, 2^W - 1]$ shares the most significant bit (MSB) with x, so $a = y \oplus x$ has that bit cleared, ensuring $0 < a < x$ and $a \oplus x = y > x$. The mapping $y \mapsto y \oplus x$ is a bijection between $(x, 2^W - 1]$ and $\{a \in (0, x) : a \oplus x > x\}$, so the count is $(2^W - 1) - x$.

- **Count of a such that $a + x = a \oplus x$ $(0 \le a \le x)$:**

$$2^{\mathrm{zeros}(x)}$$

  where $\mathrm{zeros}(x)$ is the count of zero bits in the binary representation of x (within width W). *Reason:* $a + x = a \oplus x \iff a \,\&\, x = 0$ (no carries). Since the MSB of x is 1, a must have MSB 0, which automatically guarantees $a < x$. Each of the $\mathrm{zeros}(x)$ bit positions where x has 0 can independently be chosen as 0 or 1 in a. If strictly positive $a > 0$ is required, the count is $2^{\mathrm{zeros}(x)} - 1$.

All Subarray ORs/ANDs/GCDs $O(N \log(\max A))$

```
vector<pair<ll, int>> groups, nxt;
for (int i = 0; i < n; i++)
{
    nxt.clear();
    nxt.emplace_back(a[i], i);
    for (const auto &[val, l] : groups)
    {
        ll cur = op(val, a[i]);
        if (cur == nxt.back().first)
            continue;
        nxt.emplace_back(cur, l);
    }
    swap(groups, nxt);
    // groups now holds all distinct `op` values of subarrays ending at i
    // for example: this counts how many subarrays have op == x
    for (int j = 0; j < (int)groups.size(); j++)
    {
        int lo = (j + 1 < (int)groups.size()) ? groups[j + 1].second + 1 : 0;
        mp[groups[j].first] += groups[j].second - lo + 1;
    }
}
```

Gray Code Generation $O(N \cdot 2^N)$

```
vector<string> grayCode(int n)
{
    vector<string> result;
    for (int i = 0; i < (1 << n); i++)
    {
        int gray = i ^ (i >> 1);
        string code = "";
        for (int j = n - 1; j >= 0; j--)
            code += ((gray >> j) & 1) ? '1' : '0';
        result.push_back(code);
    }
    return result;
}
```

5.1 Range Bit Counting

Calculating $\&, |, \oplus$ over the range $[l, r]$ by counting the numbers with the k-th bit set.

Count k-th bit set in range $O(\log(\max A))$

```
auto solve = [&](const ll &n, const int &k)
{
    ll m = max(n + 1, 0LL);
    return ((m >> (k + 1)) << k) + max(0LL, (m & ((2LL << k) - 1)) - (1LL << k));
};
ll andRes{}, orRes{}, xorRes{};
for (int b = 0; b < 62; b++)
{
    ll c = solve(r, b) - solve(l - 1, b); // numbers in [l, r] with bit b set
    andRes |= (1LL << b) * (c == r - l + 1);
    orRes  |= (1LL << b) * (c > 0);
    xorRes |= (1LL << b) * (c & 1);
}
```

6 Bitwise Operations over Subsequences

Notation
cnt[b]: elements with b-th bit set
$z = n - \text{cnt}[b]$: elements with b-th bit cleared

6.1 XOR Operations

- **Sum of XOR of all non-empty subsequences** ($n \geq 1$; $\bigvee$ is the bitwise OR):

$$2^{n-1} \cdot \left(\bigvee_{i=1}^{n} a_i \right)$$

- **Sum of XOR of subsequences of length k:**

$$\sum_{b=0}^{\infty} (2^b) \cdot \sum_{j \text{ odd}} \binom{\text{cnt}[b]}{j} \binom{z}{k-j}$$

Sum of XOR Subsequences $O(N)$

```
ll totOr = 0;
for (const ll &x : vc)
    totOr |= x;
Mint res = power(2_m, n - 1) * totOr;
```

Sum of XOR Subsequences of length k $O(N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;
Mint res = 0;
for (int b = 0; b < maxBits; b++)
{
    int z = n - cnt[b];
    Mint ways = 0;
    for (int j = 1; j <= min(k, cnt[b]); j += 2)
        ways += nCr(cnt[b], j) * nCr(z, k - j);
    res += ways * (1ULL << b);
}
```

Sum of XOR Subsequences for All k (NTT) $O(N \log N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;
Mint inv2 = 1_m / 2_m;
vector<Mint> ans(n + 1, 0);
for (int b = 0; b < maxBits; b++)
{
    int z = n - cnt[b];
    vector<Mint> P(cnt[b] + 1), Q(z + 1);
    for (int i = 0; i <= cnt[b]; i++)
        P[i] = (i & 1 ? -nCr(cnt[b], i) : nCr(cnt[b], i));
    for (int i = 0; i <= z; i++) Q[i] = nCr(z, i);
    vector<Mint> C = convolute(P, Q);
    for (int k = 1; k <= n; k++)
    {
        Mint oddWays = (nCr(n, k) - C[k]) * inv2;
        ans[k] += oddWays * (1ULL << b);
    }
}
```

6.2 OR Operations

- **Sum of OR of all non-empty subsequences:**

$$\sum_{b=0}^{\infty}(2^b)\cdot(2^n-2^z)$$

- **Sum of OR of subsequences of length k:**

$$\sum_{b=0}^{\infty}(2^b)\cdot\left(\binom{n}{k}-\binom{z}{k}\right)$$

Sum of OR Subsequences $O(N\log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;
vector<Mint> p2(n + 1, 1);
for (int i = 1; i <= n; i++)
    p2[i] = (p2[i - 1] * 2);
Mint res = 0;
for (int b = 0; b < maxBits; b++)
{
    int z = n - cnt[b];
    res += (1ULL << b) * (p2[n] - p2[z]);
}
```

Sum of OR Subsequences of length k $O(N\log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;
vector<Mint> p2(n + 1, 1);
for (int i = 1; i <= n; i++)
    p2[i] = (p2[i - 1] * 2);
Mint res = 0;
for (int b = 0; b < maxBits; b++)
{
    int z = n - cnt[b];
    res += (1ULL << b) * (nCr(n, k) - nCr(z, k));
}
```

6.3 AND Operations

- **Sum of AND of all non-empty subsequences:**

$$\sum_{b=0}^{\infty}(2^b)\cdot(2^{\text{cnt}[b]}-1)$$

- **Sum of AND of subsequences of length k:**

$$\sum_{b=0}^{\infty}(2^b)\cdot\binom{\text{cnt}[b]}{k}$$

Sum of AND Subsequences $O(N\log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;
vector<Mint> p2(n + 1, 1);
for (int i = 1; i <= n; i++)
    p2[i] = (p2[i - 1] * 2);

Mint res = 0;
for (int b = 0; b < maxBits; b++)
    res += (1ULL << b) * (p2[cnt[b]] - 1_m);
```

Sum of AND Subsequences of length k $O(N\log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;
vector<Mint> p2(n + 1, 1);
for (int i = 1; i <= n; i++)
    p2[i] = (p2[i - 1] * 2);

Mint res = 0;
for (int b = 0; b < maxBits; b++)
    res += (1ULL << b) * nCr(cnt[b], k);
```

7 Bitwise Operations over Subarrays

7.1 XOR Operations

Sum of XOR of all subarrays

Formula: $\sum_{b=0}^{\infty}(2^b) \cdot f_b(n)$, where $f_b(n)$ counts subarrays where b-th bit contributes (parity of prefix counts).

Sum XOR All Subarrays $O(N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;

ll res = 0;
for (int b = 0; b < maxBits; b++)
{
    ll c0 = 1, c1 = 0, pref = 0;
    for (int i = 0; i < n; i++)
    {
        pref ^= ((vc[i] >> b) & 1);
        res += (1ULL << b) * (pref ? c0 : c1);
        c1 += !!pref;
        c0 += !pref;
    }
}
```

Sum of XOR of subarrays of length k

Formula: $\sum_{b=0}^{\infty}(2^b) \cdot f_b(n, k)$ using prefix XOR in $O(N)$.

Sum XOR Subarrays Length k $O(N)$

```
int n = vc.size();
if (n < k)
    return 0;
ll res = 0;
vector<ll> pref(n + 1, 0);
for (int i = 0; i < n; i++)
    pref[i + 1] = pref[i] ^ vc[i];
for (int i = 0; i + k <= n; i++)
    res += (pref[i + k] ^ pref[i]);
```

7.2 OR Operations

Sum of OR of all subarrays

Formula: $\sum_{b=0}^{\infty}(2^b) \cdot g_b(n)$ based on distance from last occurrence.

Sum OR All Subarrays $O(N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;

ll res = 0;
for (int b = 0; b < maxBits; b++)
{
    ll last = -1;
    for (int i = 0; i < n; i++)
    {
        if ((vc[i] >> b) & 1)
            last = i;
        if (last != -1)
            res += (1ULL << b) * (last + 1);
    }
}
```

Sum of OR of subarrays of length k

Formula: $\sum_{b=0}^{\infty}(2^b) \cdot g_b(n,k)$

Sum OR Subarrays Length k $O(N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;

ll res = 0;
for (int b = 0; b < maxBits; b++)
{
    ll cnt = 0;
    ll total = n - k + 1;
    for (int i = 0; i < n; )
    {
        if ((vc[i] >> b) & 1)
        {
            i++;
            continue;
        }
        int j = i;
        while (j < n && !((vc[j] >> b) & 1))
            j++;
        int len = j - i;
        if (len >= k)
            cnt += (len - k + 1);
        i = j;
    }
    res += (1ULL << b) * (total - cnt);
}
```

7.3 AND Operations

Sum of AND of all subarrays

Formula: $\sum_{b=0}^{\infty}(2^b) \cdot h_b(n)$ counting consecutive segments with bit set.

Sum AND All Subarrays $O(N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;

ll res = 0;
for (int b = 0; b < maxBits; b++)
{
    ll cnt = 0;
    for (int i = 0; i < n; i++)
    {
        if ((vc[i] >> b) & 1)
            res += (1ULL << b) * ++cnt;
        else
            cnt = 0;
    }
}
```

Sum of AND of subarrays of length k

Formula: $\sum_{b=0}^{\infty}(2^b) \cdot h_b(n, k)$

Sum AND Subarrays Length k $O(N \log(\max A))$

```
ll maxA = *max_element(vc.begin(), vc.end());
int maxBits = maxA ? __lg(maxA) + 1 : 1;

ll ans = 0;
for (int b = 0; b < maxBits; b++)
{
    ll cnt = 0;
    for (int i = 0; i < n; )
    {
        if (!((vc[i] >> b) & 1))
        {
            i++;
            continue;
        }
        int j = i;
        while (j < n && ((vc[j] >> b) & 1))
            j++;
        int len = j - i;
        if (len >= k)
            cnt += (len - k + 1);
        i = j;
    }
    ans += cnt * (1ULL << b);
}
```

8 Range Queries & Bitwise Operations

Segment Tree with Bit Frequencies

Instead of maintaining the actual values in the segment tree, maintain an array `cnt[b]` for each node, representing the **number of elements in the range where the b-th bit is set**.
This directly provides the frequencies needed for the Subsequence formulas in Section 6.

8.1 Lazy Propagation Updates

When performing range updates, processing each bit independently allows updates in $O(\text{bits})$ time:

- **Range XOR by X:** For each bit b, if the b-th bit of X is 1, it flips the b-th bit of every element in the range:

$$\text{newCnt}[b] = \text{lengthOfRange} - \text{cnt}[b]$$

- **Range OR by X:** For each bit b, if the b-th bit of X is 1, it forces the b-th bit of every element to become 1:

$$\text{newCnt}[b] = \text{lengthOfRange}$$

- **Range AND by X:** For each bit b, if the b-th bit of X is 0, it forces the b-th bit of every element to become 0:

$$\text{newCnt}[b] = 0$$

8.2 Querying Subsequences over a Range $[L, R]$

To answer queries like *"What is the sum of XOR/OR/AND of all subsequences in range $[L, R]$?"*:

1. Query the segment tree to get the combined `cnt[b]` array for the range $[L, R]$.
2. Let $len = R - L + 1$.
3. For XOR, apply: `res += (1ULL << b) * (cnt[b] > 0 ? p2[len - 1] : 0)`.
4. For AND/OR, use the respective formulas from Section 6 using len and $\text{cnt}[b]$.

8.3 Querying Subarrays over a Range $[L, R]$

Because bitwise $\&, |, \gcd$ are monotonic and take at most $\approx \log_2(\max A)$ distinct values along prefixes and suffixes, each Segment Tree node maintains:

- `tot`: Sum of bitwise operations over all subarrays completely within $[l, r]$.
- `pref`: Vector of pairs $(\text{val}, \text{count})$ of distinct prefixes starting from l.
- `suff`: Vector of pairs $(\text{val}, \text{count})$ of distinct suffixes ending at r.

Range Subarray XOR Sum via Prefix XOR: A subarray $A[i \dots j]$ has bit b set $\iff$ $P[j]$ and $P[i-1]$ have differing b-th bits. For a range $[L, R]$, let $c_1[b]$ be the count of prefixes in $\{P[L-1] \dots P[R]\}$ with bit b set, and $c_0[b] = (R - L + 2) - c_1[b]$. The total XOR sum is:

$$\sum_{b=0}^{\infty} (2^b) \cdot c_0[b] \cdot c_1[b]$$

Maintained in $O(\log(\max A))$ via prefix bit counts without segment tree node merging.

Segment Tree Subarray Node Merge $O(\log^2(\max A))$

```
struct Node
{
    ll tot;
    vector<pair<ll, ll>> pref, suff;
    Node() { tot = 0; }
    Node(ll val)
    {
        tot = val;
        pref.emplace_back(val, 1);
        suff.emplace_back(val, 1);
    }
};

Node merge(const Node &leftNode, const Node &rightNode)
{
    Node res;
    res.tot = leftNode.tot + rightNode.tot;
    for (const auto &[u, cu] : leftNode.suff)
    {
        for (const auto &[v, cv] : rightNode.pref)
            res.tot += op(u, v) * cu * cv;
    }

    res.pref = leftNode.pref;
    ll fullL = leftNode.pref.back().first;
    for (const auto &[v, cv] : rightNode.pref)
    {
        ll cur = op(fullL, v);
        if (cur == res.pref.back().first)
            res.pref.back().second += cv;
        else
            res.pref.emplace_back(cur, cv);
    }

    res.suff = rightNode.suff;
    ll fullR = rightNode.suff.back().first;
    for (const auto &[u, cu] : leftNode.suff)
    {
        ll cur = op(u, fullR);
        if (cur == res.suff.back().first)
            res.suff.back().second += cu;
        else
            res.suff.emplace_back(cur, cu);
    }
    return res;
}
```

9 String Algorithms

Palindrome Parity Mask & Rearrangeable Substrings

Represent character frequency parities with a bitmask (c-th bit is 1 if character count is odd, 0 if even).

> A string can be rearranged into a palindrome $\iff$ popcount(mask) ≤ 1.

Prefix Parity Mask: Let $P[i]$ be the prefix XOR mask of $S[0 \dots i]$. A substring $S[l \dots r]$ is rearrangeable into a palindrome $\iff$ popcount$(P[r] \oplus P[l-1]) \leq 1$.

Count Rearrangeable Palindromic Substrings $O(|\Sigma| \cdot N \log N)$

```
ll mask = 0, res = 0;
map<int, int> cnt;
cnt[0] = 1;
for (const char &c : str)
{
    mask ^= (1 << (c - 'a'));
    res += cnt[mask];
    for (int b = 0; b < 26; b++)
        res += cnt[mask ^ (1 << b)];
    cnt[mask]++;
}
```

String Periodicity & KMP Border Property

Problem 1 — Find the shortest base T such that $S = T^k$: Compute $k = n - \pi[n-1]$. If $n \bmod k = 0$ and $\pi[n-1] > 0$, then $T = S[0 \dots k-1]$ and $S = T^{n/k}$. Otherwise, S has no such repetition, and the code sets period $= n$ (i.e. S itself).

Smallest Repeating Period $O(N)$

```
int k = n - b[n]; // b[n] is PI[n - 1]
int period = (b[n] > 0 && n % k == 0) ? k : n;
string root = str.substr(0, period);
int repetitions = n / period;
```

Application — Shortest string with exactly k copies of t

Consecutive occurrences of t can overlap by $\pi[n-1]$ characters (its longest border). So each new occurrence after the first only requires appending $t\,[\pi[n-1] \dots n-1]$ — the last $n - \pi[n-1]$ characters. The shortest such string has length $n + (k-1)(n - \pi[n-1])$.

Shortest string with exactly k copies of t

```
preprocessKMP(str);
int l = b[n];
cout << str;
for (int i{}; i < k - 1; i++)
    cout << str.substr(l);
```

Cyclic Shift Equivalence & Minimal Rotation

Core question: Are two strings A and B the same string written on a circle, just read from a different starting point? Two tools answer this, depending on the situation:

> **method 1 — Pairwise check:** For $|A| = |B|$, B is a rotation of $A \iff B$ appears as a substring in $A + A$. The match index i is also the minimum left shift to transform A into B, and $(n - i) \bmod n$ is the equivalent right shift. Use when comparing two specific strings.

> **method 2 — Canonical form (`minRotation`):** Given a circular string, read it from every possible starting point and return the lexicographically smallest result. Since all rotations of the same circle produce the same canonical string, `minRotation`$(A) =$ `minRotation`$(B) \iff A$ and B are rotations of each other. Use when grouping or hashing many circular strings — compute once per string, then use as a map key instead of doing $O(K^2)$ pairwise checks.

Duval's Algorithm computes `minRotation` in $O(N)$ time by doubling the string and finding the lexicographically smallest length-n substring via Lyndon factorization.

Minimal String Rotation (Duval's Algorithm) $O(N)$

```cpp
string minRotation(string str)
{
    str += str;
    int n = str.size(), i = 0, res = 0;
    while (i < n / 2)
    {
        res = i;
        int j = i + 1, k = i;
        while (j < n && str[k] <= str[j])
        {
            if (str[k] < str[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) // Skip suboptimal candidate starting positions
            i += j - k;
    }
    return str.substr(res, n / 2);
}
```

Greedy Concatenation Ordering

To arrange strings such that their concatenation is lexicographically smallest (or largest), standard element-wise comparison fails.

> **Comparator:** $A \prec B \iff A + B < B + A$ (this defines a strict weak ordering).

Sort by Concatenation Order $O(N \log N \cdot L)$

```cpp
sort(vc.begin(), vc.end(), [](const string &a, const string &b)
{ return a + b < b + a;});
```

10 Greedy & Poset Principles

Dilworth's Theorem & Sequence Partitions

In any finite partially ordered set:

$$\text{Size of Maximum Antichain} = \text{Minimum Number of Chains to Cover Set}$$

> The minimum number of non-decreasing subsequences needed to partition an array equals the length of the **Longest Strictly Decreasing Subsequence**.

11 Chess Problems

Knights on a Chessboard ($n \times m$)

Ways two knights attack each other (for $n, m \geq 2$; the formula counts 2×3 and 3×2 rectangles):

$$2 \cdot (n-1) \cdot (m-2) + 2 \cdot (n-2) \cdot (m-1)$$

Ways two knights do NOT attack each other:

$$\frac{n \cdot m \cdot (n \cdot m - 1)}{2} - (2 \cdot (n-1) \cdot (m-2) + 2 \cdot (n-2) \cdot (m-1))$$

12 Geometry

Manhattan to Chebyshev Rotation & 2D Median

Applying the coordinate rotation $(x', y') = (x+y, x-y)$ transforms 2D Manhattan distance into Chebyshev distance:

$$|x_1 - x_2| + |y_1 - y_2| = \max(|x'_1 - x'_2|, |y'_1 - y'_2|)$$

This decouples the two coordinates, turning 2D distance bounds into independent 1D range queries.
Optimal Meeting Point (Minimizing Sum of Distances):

- **1D Line:** The optimal point is the **median** of the coordinates.
- **2D Manhattan Distance:** Independent for axes. Optimal point is $(\text{median}(X), \text{median}(Y))$.
- **2D Chebyshev Distance** (note $\max(|\Delta x|, |\Delta y|) = \frac{|\Delta u| + |\Delta v|}{2}$): Rotate to Manhattan $(u, v) = (x + y, x - y)$, find the medians U_{opt} and V_{opt}. Since valid original grid points require $u \equiv v \pmod 2$, if U_{opt} and V_{opt} have different parities, test the 4 adjacent points with matching parities, then rotate back.

Pick's Theorem for Lattice Polygons

For any simple polygon with vertices on integer grid coordinates:

$$\text{Area} = I + \frac{B}{2} - 1$$

where I is the number of strictly interior points, and $B = \sum_{i=1}^{n} \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$ is the number of boundary lattice points.

Tiling Grid into a Square

Number of $n \cdot m$ grid copies needed to form a square (without intersection nor changing orientation):

$$\frac{n \cdot m}{\gcd(n, m)^2}$$